

VIDEOJUEGO DRAGON BALL – ENTRENANDO AL ESTILO DE ROSHI

Momento III: Informe Final

Autor: Esteban Rodríguez Monsalve

Fecha: 13 de Julio de 2025

Repositorio: [GitHub](#)

Equipo: #14

Introducción

El videojuego **Dragon Ball – Entrenando al estilo de Roshi** está inspirado en la serie clásica de Dragon Ball, específicamente en el episodio 18: *“Entrenando al estilo de Roshi”*. En este capítulo, Goku y Krillin realizan un entrenamiento riguroso bajo la supervisión del Maestro Roshi, enfrentando tareas cotidianas con un pesado caparazón que intensifica el esfuerzo físico.

Basado en esta premisa, el juego se desarrolla como un **plataformero 2D**, dividido en **tres niveles** con grados de dificultad crecientes y físicas únicas para cada uno, que modelan peso, trayectoria de lanzamientos y resistencia de fluidos.

El proyecto está construido bajo los principios de **Programación Orientada a Objetos (POO)**, hace uso de **memoria dinámica** cuidadosamente gestionada y combina herencias propias y herramientas clave de **Qt Framework**, junto con **C++**, para asegurar una arquitectura escalable, modular y eficiente.

Modelos Físicos Implementados

El juego integra **tres modelos físicos reales**, distintos del movimiento rectilíneo básico:

1. Peso Dinámico (Nivel 1)

- El caparazón afecta la velocidad y salto de Goku:
 - Velocidad reducida (factor 0.7)

- Salto limitado (factor 0.8)
- Gravedad ajustada mediante `Fisica::aplicarGravedad()` y colisiones con `Fisica::verificarColisionSuelo()`

2. Trayectoria Parabólica (Nivel 2)

- Se implementa la física de proyectiles para lanzar botellas de leche:
 - Cálculo de ángulo y fuerza usando `Fisica::calcularTiroParabolico()` y `Goku::disparar()`
 - Influencia del caparazón para dificultar precisión

3. Resistencia de Fluidos (Nivel 3)

- Goku nada en un río enfrentando resistencia del agua:
 - `Fisica::aplicarFisicaAgua()` regula flotación y velocidad
 - El caparazón otorga flotabilidad adicional (`Caparazon::factorFlotacion`)

Resumen de Clases y Dinámica del Sistema

Clase Base: Personaje

- **Función:** Define atributos y métodos comunes para cualquier entidad jugable.
- **Atributos clave:** vidas, velocidad, posX, posY.
- **Métodos:** `mover()`, `saltar()`, `recibirDaño()`, `actualizarAnimacion()`.
- **Relaciones:** Es heredada por Goku.

Clase: Goku

- **Función:** Representa al jugador principal.
- **Herencia:** Deriva de Personaje.
- **Atributos adicionales:** tirosDisponibles, esferasRecolectadas, burbujasRecolectadas.
- **Métodos:** disparar(), nadar(), recolectar(), activarInvulnerabilidad().
- **Relaciones:** Usa Caparazon y Fisica para ajustar sus capacidades físicas y reacciones.

Clase: Caparazon

- **Función:** Modela el peso que limita la velocidad, salto y flotación de Goku.
- **Atributos:** nivelPeso, factorVelocidad, factorSalto, factorFlotacion.
- **Método:** actualizarPeso(nivel) permite variar dinámicamente la dificultad.

Clase: Item (abstracta)

- **Función:** Base genérica para todos los ítems recolectables.
- **Herencia:** EsferaDragon, CajaLeche y Burbuja derivan de ella.
- **Método virtual:** efecto() se redefine según la subclase.

Subclases de Item:

- **EsferaDragon:** Incrementa el puntaje y avanza el progreso.
- **CajaLeche:** Otorga tiros extra para lanzar en Nivel 2.
- **Burbuja:** Añade vidas extra (nivel 3).

Clase: Obstaculo

- **Función:** Representa amenazas dinámicas (peces, escombros).
- **Comportamiento:** Se mueven automáticamente y reducen vidas al impactar a Goku.

Clase: Plataforma

- **Función:** Superficies de apoyo (fijas o móviles).
- **Tipos:** FIJA, MOVIL, ASCENSOR.
- **Métodos:** Movimiento automático, sacudidas, colisiones.

Clase: Fisica

- **Función:** Unifica todos los cálculos físicos del juego.
- **Métodos clave:**
 - `aplicarGravedad()` (Nivel 1)
 - `calcularTiroParabolico()` (Nivel 2)
 - `aplicarFisicaAgua()` (Nivel 3)
 - `afectarObjeto()` (caídas de escombros)

Clase: Nivel (abstracta)

- **Función:** Plantilla para Nivel1, Nivel2 y Nivel3.
 - **Métodos virtuales:** `cargarFondo()`, `agregarItems()`, `verificarObjetivoCompleto()`.
 - **Especialización:** Cada subnivel redefine física, objetivos y obstáculos.
-

Dinámica General

El flujo de juego orquesta la interacción de todas estas clases:

- Juego maneja la escena (QGraphicsScene y QGraphicsView), cambia niveles con QStackedLayout y controla paneles de información (QWidget, QLabel).
- Goku interactúa con Física para moverse, saltar, nadar y lanzar objetos.
- Caparazon influye en todos los cálculos físicos (peso, velocidad, flotación).
- Item y sus subclases se gestionan en contenedores QList y afectan atributos de Goku con su efecto().
- Plataforma y Obstaculo agregan reto dinámico: movimiento automatizado, colisiones y daño.
- AudioManager (usa QMediaPlayer y QAudioOutput) gestiona música y efectos, vinculando eventos clave del juego.

Herencias Propias

Herencia	Propósito	Beneficio
Personaje → Goku	Reutilizar lógica de entidades jugables y permitir expansión (Krillin, Enemigos).	Extensibilidad, claridad en POO.
Item → EsferaDragon, CajaLeche, Burbuja	Abstraer recolección: polimorfismo mediante efecto().	Aumenta modularidad.

Nivel → Nivel1, Nivel2, Nivel3	Compartir lógica de escena, física y flujo.	Reduce duplicidad, facilita escalado.
---------------------------------------	---	---------------------------------------

Manejo de Dificultad

Cada nivel muestra progresión de dificultad:

- **Nivel 1:** Introduce plataformas móviles y zonas de caída.
- **Nivel 2:** Requiere dominio de fuerza y ángulo; originalmente se planteó una plataforma obligatoria inestable, pero se cambió por ajustes dinámicos de trayectoria, mejorando jugabilidad.
- **Nivel 3:** Integra física de fluidos, obstáculos móviles (peces, escombros) y mecánicas de recolección de burbujas.

Estos cambios reflejan un proceso real de abstracción de dificultad, adaptando ideas iniciales a la mejor experiencia jugable.

Herramientas de Qt Utilizadas

- **QGraphicsView & QGraphicsScene:** Motor principal de renderizado 2D.
- **QTimer:** Manejo de física, animaciones y control de eventos.
- **QPixmap:** Sprites de Goku y objetos.
- **QMediaPlayer & QAudioOutput:** Manejo de música de fondo y efectos de sonido.
- **QWidget, QDialog, QPushButton, QLabel:** Construcción de interfaz gráfica (menús, paneles de información, ventanas emergentes).
- **QStackedLayout & QHBoxLayout:** Organización modular de vistas de juego y menús.

Contenedores Qt

Clase Qt	Usada en	Propósito
QGraphicsScene	Nivel	Manejo de la escena
QGraphicsPixmapItem	Personaje, Item	Mostrar los sprites
QObject	Goku, AudioManager	Señales y slots, timers
QWidget	Juego, PanelInformacion, VentanaEmergente	Interfaces y menús
QTimer	Múltiples	Animaciones, físicas y eventos
QGraphicsView	Juego	Mostrar dentro de ventana
QStackedLayout	Juego	Cambio entre vistas (menús, niveles)

Contenedores Qt:

- `QList<Item*>`: Maneja todos los objetos recolectables.
- `QList<Obstaculo*>`: Controla enemigos y obstáculos.
- `QHash<TipoSonido, QMediaPlayer*>`: Mapear efectos de sonido.

- `QSet<int>`: Almacena teclas activas para movimiento continuo.
- `QGraphicsItem*` en listas: Para verificar colisiones.

Justificación: Qt ofrece contenedores seguros y optimizados para operaciones de búsqueda, iteración y almacenamiento de objetos dinámicos. Su integración con `QObject` facilita la conexión de señales y slots.

Gestión de Memoria Dinámica

- Uso intensivo de `new` para instanciar objetos (niveles, sprites, plataformas).
 - **Problema:** `delete` manual fallaba debido a `QTimer` y señales en ejecución.
 - **Solución:** `QObject::deleteLater()` garantiza la destrucción cuando la cola de eventos finaliza, evitando referencias colgantes.
-

Animaciones y Estados

- **Sprites:** Se gestionan como `QVector<QPixmap>`.
 - **QTimer:** Controla ciclos de 120 ms para animaciones.
 - **Transiciones:** `Goku::cambiarEstado()` reinicia el índice y sincroniza la animación.
 - **Estados:** Quieto, Caminando (izq/der), Nadando, Lanzando.
-

Interacción de Clases en Ejecución

Ejemplo:

- Goku se mueve (`Personaje::mover()`), colisiona con Plataforma (`Fisica::verificarColisionSuelo()`).
- Recolecta CajaLeche (`Item::efecto()`), que suma tiros (`Goku::aumentarTiros()`).
- En Nivel 2, usa `disparar()` y `calcularTiroParabolico()` para lanzar.
- Si choca con Obstaculo, recibe daño (`Personaje::recibirDaño()`).
- Juego actualiza `PanelInformacion` vía señales y slots.

Cambios de Momentos I y II a la Versión Final

Aspecto	Momento I	Momento II	Final
Competencia con Krillin	Incluida	Removida	No implementada
Plataformas destruyéndose tras colisión	Planeadas	Revisadas	Se removió por sobrecarga de dificultad
Plataforma inestable Nivel 2	Planeada	Revisada	Se cambió por ajuste de trayectoria dinámica
Herencia Enemigo	Planeada	Parcial	No implementada (preparada para expansión)
Física de fluidos	Descrita	Refinada	Implementada con zona de agua y resistencia

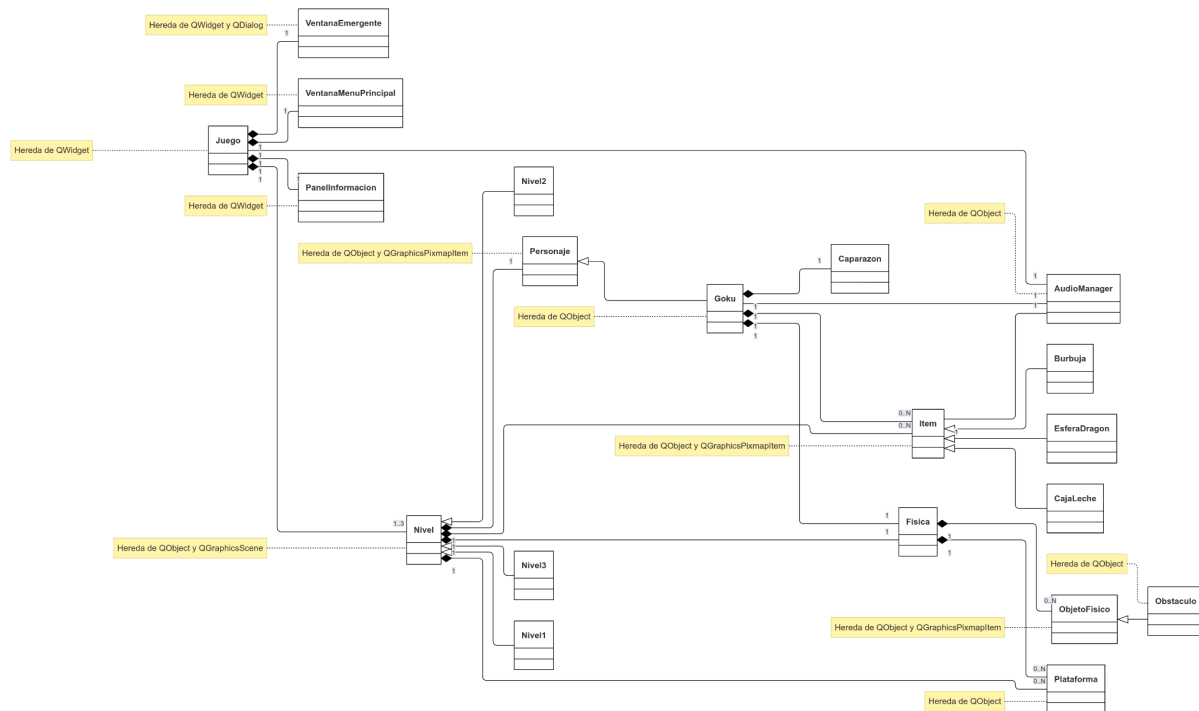
Uso de Excepciones

El sistema lanza excepciones controladas al detectar errores como carga fallida de recursos de audio o imágenes, validación de rutas y estados no válidos. Esto se combina con QDebug para seguimiento en consola.

Dificultades y Soluciones

- Manejo de delete y sincronización con QTimer. Solución: uso de deleteLater() para evitar accesos fuera de tiempo.
 - Redimensionar imágenes y animaciones.
 - Implementar cambios de estado fluidos en Goku y plataformas.
 - Sincronizar el movimiento del personaje con plataformas en Nivel 1.
 - Afinar trayectorias y fuerza de lanzamiento en Nivel 2.
 - Configurar física de agua y colisiones precisas en Nivel 3.
-

Diagrama de Clases simplificado



Conclusión

Este proyecto demuestra dominio de **POO**, herencias, físicas y optimización de recursos Qt. Cumple con:

- 3 físicas reales y diferenciadas.
- Herencia propia y polimorfismo efectivo.
- Uso de contenedores Qt.
- Animación controlada con `QTimer`.
- Memoria dinámica y manejo seguro de destrucción.

El resultado: un juego progresivo, retador y fiel al espíritu de Dragon Ball.

